Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Program: Software Engineering

Operating Systems

Laboratory Work No. 4

The xv6 Shell from the Outside

Track B: Build Your Own Small OS (xv6)

Done by:
st. gr. FAF-241
Sitarciuc Dmitri

Checked by:
Reițman Patricia

Chișinău – 2026

Laboratory Work No. 4 – The xv6 Shell from the Outside

1. Introduction

The purpose of this laboratory work was to find out which jobs belong to the shell and which belong to the program it starts. I compared the same commands in Bash and in the xv6 shell, wrote a program `args` that prints exactly the arguments it receives, and read the source of the xv6 shell, `user/sh.c`.

The work was done in GitHub Codespaces: the Bash side in Ubuntu, the xv6 side in QEMU, using the same xv6 tree as in Lab 2. The Linux commands were run from a script that prints each command with `$` before its output.

2. Checking sleep for the Current xv6

First I checked that the `sleep` program from Lab 2 still matches this xv6 version:

```
$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
$ grep -n "_sleep" Makefile
142:    $U/_sleep\
```

Figure 2.1 – pause() in user/user.h and sleep in UPROGS

The waiting system call is declared as `pause(int)`, and `user/sleep.c` already calls `pause(atoi(argv[1]))`, so no change was needed. The program is still listed in `UPROGS`.

Test in xv6:

```
$ sleep 20
$ sleep
usage: sleep <ticks>
$
```

Figure 2.2 – sleep with and without an argument in xv6

`sleep 20` waited briefly and returned to the prompt, and `sleep` without an argument printed the usage message, so both cases work.

3. bash versus the xv6 Shell

I ran the same commands in Bash (in a folder with a copy of `README`) and at the xv6 `$` prompt.

3.1. Comparison Table

Command	Feature	bash	xv6
`echo hello > f ; cat < f`	redirection	`hello`	`hello`
`ls | grep READ`	pipe	`README`	`README 2 2 2441`
`(echo a; echo b) | wc`	grouping + pipe	`2 2 4`	`2 2 4`
`echo one ; echo two`	sequence	`one` / `two`	`one` / `two`
`echo bg &`	background	`bg` (job number shown)	`bg`, printed after the next `$` prompt
`ls *`	globbing	`README f`	`ls: cannot open *`
`echo $HOME`	variables	`/home/codespace`	`$HOME`
`echo "a b"`	quotes	`a b`	`"a b"`
`mkdir d ; cd d ; ls ..`	command lookup	`README d f`	`exec ls failed` (typed as three separate commands)
arrow up, Tab	history / completion	supported (Readline)	not supported: `getcmd()` reads a plain line with `gets()` (sh.c, line 139), so keys are not interpreted

Table 3.1 – The same commands in Bash and in the xv6 shell

Redirection, pipes, grouping, sequences and background jobs work in both shells. The differences appear in globbing, variables and quotes: Bash changes `*`, `$HOME` and `"a b"` before running `echo` or `ls`, while xv6 passes them on unchanged.

In xv6, after `cd d` the plain `ls` fails, but a relative path works:

```
$ mkdir d
$ cd d
$ ls ..
exec ls failed
$ ../ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
...
f              2 26 6
d              1 27 32
```

Figure 3.1 – ls fails after cd d in xv6, ../ls works (listing shortened)

3.2. Observations

1. Both shells can run programs, redirect input and output, build pipes, run command lists with `;`, group with parentheses, start background jobs with `&` and change directory. Only Bash also does globbing, variable expansion, quote removal, `PATH` search, history and Tab completion.
2. `ls` fails after `cd d` because the xv6 shell has no `PATH`. It runs `exec` on the name exactly as typed, relative to the current directory. All programs live in `/`, so from `d` they must be called as `../ls` (or `/ls`).
3. The missing feature I would notice most is `PATH` lookup, because every command breaks as soon as you leave `/`. It would also be one of the easiest to add: if `exec` fails and the name has no `/`, try again with `/` in front.

3.3. Interesting Command Output

`echo bg &` in xv6 printed the prompt `$` first and `bg` after it. The shell did not wait for the background child, so it printed the prompt at once, and the child's output arrived a moment later. Also, `../ls ..` shows the file `f` with size 6, which is exactly `hello` plus a newline written by the redirection earlier.

4. Who Expands *? The args Program

To see exactly what a program receives, I wrote `args`, which prints `argc` and every `argv[i]` in brackets without changing anything.

4.1. xv6 Version

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

Figure 4.1 – user/args.c

I added `$U/_args\` to `UPROGS` (line 143, after `_sleep`) and rebuilt xv6. Results:

```
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
$ args one   two      three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

Figure 4.2 – Arguments received by args in xv6

4.2. Linux Version

The Linux version is the same code with `#include <stdio.h>` and `return 0`, compiled with `gcc -o args args_linux.c`. The folder contained three `.c` files: `args_linux.c`, `test1.c` and `test2.c`.

```
$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [test1.c]
argv[3] = [test2.c]
argv[4] = [x y]
$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/codespace]
$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
$ ./args one   two      three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
```

Figure 4.3 – Arguments received by args under Bash on Linux

4.3. Observations

1. For `args *.c "x y"` the xv6 program received 4 arguments: its name, the literal `*.c`, and `"x y"` cut at the space into `"x` and `y"`, quotes included. Under Bash the program received 5: `*.c` became three file names and `"x y"` became one argument `x y` without quotes.
2. The same C code got different `argv` depending only on the shell, so the shell does the expansion, not the program and not the kernel. Bash expands `*` and `$HOME` and removes quotes before calling `exec`; the xv6 shell only splits the line on spaces and passes the words on.
3. Extra spaces between words do not reach the program in either shell. `one two three` gave 4 arguments with no spaces inside them in both xv6 and Bash, because both tokenizers treat any run of spaces as one separator.

5. Reading the xv6 Shell

The whole xv6 shell is one file:

```
$ wc -l user/sh.c
499 user/sh.c
$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 5.1 – Size of sh.c and the command kinds it defines

`grep -n "case " user/sh.c` showed where each kind is executed in `runcmd()` (lines 75 `EXEC`, 83 `REDIR`, 93 `LIST`, 101 `PIPE`, 125 `BACK`) and where the tokenizer recognizes special characters (lines 286–294: `|`, `(`, `)`, `;`, `&`, `<`, `>`, plus `>>` on line 397).

The handling of `cd` in the main loop:

```
while (getcmd(buf, sizeof(buf)) >= 0) {
  char *cmd = buf;
  ...
  if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
    // Chdir must be called by the parent, not the child.
    cmd[strlen(cmd) - 1] = 0; // chop \n
    if (chdir(cmd + 3) < 0)
      fprintf(2, "cannot cd %s\n", cmd + 3);
  } else {
    if (fork1() == 0)
      runcmd(parsecmd(cmd));
    wait(0);
  }
```

Figure 5.2 – Special handling of cd in the main loop of sh.c (lines 160–175)

Searching for expansion features:

```
$ grep -c "\*" user/sh.c
116
$ grep -n "glob\|getenv\|PATH" user/sh.c
matches: 0
$ grep -n '\$' user/sh.c
137:  write(2, "$ ", 2);
```

Figure 5.3 – Searching sh.c for globbing, variables and PATH

5.1. Observations

1. The five command kinds are `EXEC` (run a program), `REDIR` (`<`, `>`, `>>`), `PIPE` (`|`), `LIST` (`;`) and `BACK` (`&`). Parentheses only group commands during parsing; they do not need their own kind. `MAXARGS 10` also limits a command to 10 arguments.
2. `cd` is checked in the main loop and run by the shell itself with `chdir`, before any `fork`. Every other command runs in a child created by `fork1()`. If `cd` ran in a child, only the child's directory would change and the shell would stay where it was. The comment in the code says exactly this.
3. `*` appears 116 times, but all of them come from C syntax (pointer declarations like char * and dereferences like *cmd) and comments; there is no `glob`, `getenv` or `PATH` anywhere. The only `$` is the prompt on line 137. This is why in Parts 3 and 4 xv6 passed `*`, `$HOME` and quotes through unchanged and could not find `ls` from inside `d`.

6. Conclusion

The xv6 shell already has the core mechanisms of a Unix shell: running programs with `fork` and `exec`, redirection, pipes, command lists, grouping and background jobs, all in 499 lines. What it lacks are the conveniences Bash adds before a program starts: globbing, variables, quote removal and `PATH` search.

The `args` experiment made this visible. The same C program received `*.c`, `$HOME` and quote characters literally under xv6, but file names, `/home/codespace` and clean words under Bash. So these transformations are done by the shell, not by the kernel or the program.

Reading `sh.c` explained every result: five command kinds, `cd` handled in the parent before `fork`, and no code for globbing, variables or `PATH`.